SmartBet Online API Routes

A concise integration reference for the online betting website. Routes below are taken from the supplied API guide; deployed behavior has not been independently tested. Example data is illustrative.

API base URLs

Configure these addresses for each environment. The values below are **configuration names, not actual server addresses**.

Configuration name	Purpose
`backendUrl`	Base URL of the online backend, for example `https://<backend-host>`; account, ticket, payment, and settings routes use its `/api` prefix
`gamesBaseUrl`	Base URL of the prematch games and odds service
`liveGamesBaseUrl`	Base URL of the live Socket.IO service for games, scores, odds, and match status

The integrating developer must supply the correct addresses for their environment. No particular local port, domain, or deployment host is required by this document.

URL construction: `backendUrl + /api/Account/Login` for online endpoints; `gamesBaseUrl + /` for the prematch feed; connect to `liveGamesBaseUrl` for Socket.IO events. Do not append `/api` twice.

1. Prematch games and odds

Method	Route	Request data	Response
GET	`gamesBaseUrl/`	None	Array of matches, teams, leagues, and `MatchOdds`
GET	`/api/Matches/GetCountriesWithLeagues`	None	Countries with nested leagues
GET	`/api/Matches/GetTopMatches`	`page`, `pagesize`, optionally `includeOdds`	Top matches with selected odds
GET	`/api/Matches/GetSetCount`	None	Current set game count

Prematch game example (selected fields):

```
{
  "OriginalMatchId": 123456,
  "MatchNo": 101,
  "HomeTeamName": "Home FC",
  "AwayTeamName": "Away FC",
  "League": "Example League",
  "StartTime": "2026-11-06T18:00:00",
  "MatchOdds": [
    {"BetCategory":"1x2","BetOption":"1","Line":null,"Odd":2.1,"BookMakerId":1,"MatchOddId":9001}
  ]
}
```

The game feed is refreshed periodically (approximately every 15 minutes in the documented frontend). Match details are read from the fetched game list; no separate match-details HTTP route was identified.

2. Live games and scores

Transport / event	Address	Data received
Socket.IO connection	`liveGamesBaseUrl`	Live game event stream
`buffers`	Same connection	Full protobuf-encoded live game updates, including odds, scores, time and status
`buffer-diffs`	Same connection	Incremental updates; handling is currently disabled in the documented integration
`expired-key`	Same connection	IDs of live games to remove

Live games are delivered through Socket.IO, not a separate documented REST endpoint. The `buffers` payload contains a protobuf event with `obj` game entries. Important decoded fields are `std` (match ID), `cd` (short code), `t1.n` and `t2.n` (teams), `inf` (league), `sc.goal` (score), `tm`/`tss` (match clock), `st`/`stn` (status), and `bt` (markets/odds). The separate games HTTP feed has **not** been verified to include live matches.

3. Authentication

Method	Route	Request data	Response
POST	`/api/Account/Login`	`username`, `password`, `rememberMe`	Login result, user, session cookie
POST	`/api/Account/LogOff`	None	HTTP 200, empty body
GET	`/api/Account/CheckLogin`	Optional `strict401`	Authentication state, user and roles; anonymous may return 204/401

Login body: `{"username":"demo_user","password":"example-password","rememberMe":true}`

4. Registration and password

Method	Route	Request data	Response
GET	`/api/Account/CheckUsername`	Query: `value`	`{ "available": true/false }`
GET	`/api/Account/CheckPhone`	Query: `value`	`{ "available": true/false }`
GET	`/api/Account/CheckEmail`	Query: `value`	`{ "available": true/false }`
GET	`/api/Online/UserExists`	Query: `userName`	Boolean (exists)
POST	`/api/Account/Register`	`email`, `phoneNumber`, `userName`, `NIN`, `dob`, `FirstName`, `SurName`, `PromoCode`	Registration result and OTP expiry
GET	`/api/Account/GetOTP`	Query: `phoneNumber`	Boolean result
POST	`/api/Account/validate-otp`	`phoneNumber`, `otpCode`	OTP validation result
POST	`/api/Account/VerifyOtpAndSetPassword`	`phoneNumber`, `otpCode`, `newPassword`, `confirmPassword`	Password setup result
POST	`/api/Account/SetNewPassWord`	`NewPassword`, `ConfirmPassword`, `username`, `Code`	Password reset result

Registration body:

```
{"email":"demo@example.com","phoneNumber":"256700000000","userName":"demo_user","NIN":"EXAMPLE-NIN",
```

"dob":"1995-04-20","FirstName":"Demo","SurName":"User","PromoCode":null}

Important: Registration OTP delivery was flagged as potentially targeting a fixed test recipient in the inspected implementation. Verify before using the registration flow in production.

5. Online account

Method	Route	Request data	Response
GET	`/api/Online/UserInfo`	Session cookie	`AccountId`, `Balance`, `UserId`, `PhoneNumber`
GET	`/api/Account/Bonuses`	Session cookie	Bonus wallets and transactions
GET	`/api/Statement/User`	Query: `userid`	Statement rows

6. Tickets and bets

Method	Route	Request data	Response
POST	`/api/Ticket`	Ticket with `betData`, `totalStake`, `totalOdds`, `totalBonus`, `isLive`	Success or ticket validation errors
POST	`/api/Ticket/Booking`	Ticket payload (prematch only)	Booking result and booking code
GET	`/api/Online/MyBets`	Session cookie	Receipt/ticket history
GET	`/api/Online/OpenBetsCount`	Session cookie	Open bets count
GET	`/api/Receipt/GetById/{receiptId}`	Receipt ID in URL	Receipt details
GET	`/api/Bet/Receipt/{receiptId}`	Receipt ID in URL	Ticket selections

Ticket body (selected fields):

```
{
  "betData": [{"betCategory":"1x2","betOption":"1","bookMakerId":1,"line":null,"matchId":123456,
"matchOddId":9001,"odd":2.1,"IsLive":false}],
  "totalBonus": 0,
  "totalOdds": 2.1,
  "totalStake": 1000,
  "isLive": false
}
```

7. Deposits and withdrawals

Method	Route	Request data	Response
POST	`/api/Online/MobileMoneyDepositRequest`	`amount`, `phoneNumber`, `PromoCode`	Accepted deposit request and transaction code
GET	`/api/Online/DepositStatus/{code}`	Deposit code in URL	Status, amount, balance, update time
POST	`/api/Online/redeem`	`code`	Redeemed deposit and balance
POST	`/api/Online/WithdrawAtBranch`	`amount`	Branch withdrawal code and result
POST	`/api/Online/WithdrawRequest`	`requestId`, `amount`, optional `altPhone`	Withdrawal status, possible OTP challenge
POST	`/api/Online/ConfirmWithdrawAlt`	`challengeId`, `otpCode`	Withdrawal confirmation status

Mobile deposit body: `{"phoneNumber":"256700000000","amount":10000,"PromoCode":null}`

Mobile withdrawal body:
`{"requestId":"11111111-1111-4111-8111-111111111111","amount":10000,"altPhone":"256700000000"}`

Deposit acceptance is not payment completion; check `DepositStatus`. Withdrawal requests may return `OTP`, `PENDING`, `UNKNOWN`, `SUCCESSFUL`, `FAILED`, or `REJECTED`. Some existing withdrawal callers were reported to use an outdated payload; the request shown here is the documented current contract.

8. Online settings

Method	Route	Request data	Response
GET	`/api/CompanySettings/OnlineSettings`	None	`MinStake`, `MaxStake`, `MaxPayOut`

General request notes

Online account, ticket, and payment calls use JSON and cookie-based sessions. The games feed is a separate service. The documented HTTP client has a 20-second default timeout; payment requests can override it. HTTP 400 may contain validation errors, 401 means authentication is required, and 417 may carry ticket or payment failure details.

This document intentionally excludes backend controller names, source paths, implementation filenames, and internal administration routes.